LẬP TRÌNH WEB

CHƯƠNG 7

QUẢN LÝ PHIÊN VÀ XÁC THỰC NGƯỜI DÙNG

Giảng viên: TS. Đặng Trung Thành

Chuẩn đầu ra của chương

- Lý thuyết
  - Giải thích được cơ chế phân quyền và xác thực người dùng
  - Giải thích được cách sử dụng thông tin xác thực và phân quyền để triển khai bảo mật cho ứng dụng
  - Giải thích được cách thức lưu trữ và truy xuất thông tin phiên làm việc để tùy chỉnh trải nghiệm người dùng
- Thực hành
  - Tùy chỉnh được trang đăng nhập và tạo được trang thông tin người dùng
  - Sử dụng được thông tin tình trạng đăng nhập của người dùng để tùy chỉnh giao diện
  - Lưu trữ và truy xuất được dữ liệu phiên

Nội dung

- Xác thực người dùng
- Quản lý phiên

7.1 Xác thực người dùng

Middleware (Phần mềm trung gian)

- Các thành phần can thiệp vào tiến trình xử lý các yêu cầu (request) và phản hồi (response) để tích hợp các tính năng quan trọng như bảo mật, quản lý phiên và xác thực
- Giúp đơn giản hóa việc lập trình các chức năng web: chỉ cần tập trung vào logic nghiệp vụ
- Khai báo: trong biến MIDDLEWWARE của tập tin **settings.py**

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XframeOptionsMiddleware",
]
```

7.1 Xác thực người dùng

Middleware (Phần mềm trung gian)

- Các module:
  - **SecurityMiddleware**: cung cấp các tính năng bảo mật tăng cường phổ biến như xử lý chuyển hướng SSL và thêm các tiêu đề cho phản hồi để ngăn chặn các vụ hack thông thường
  - **SessionMiddleware**: cho phép hỗ trợ phiên và liên kết liền mạch dữ liệu về phiên với các yêu cầu (request); thêm thuộc tính session vào đối tượng request
  - **CommonMiddleware**: triển khai nhiều tính năng nhỏ, như từ chối yêu cầu (từ danh sách DISALLOWED_USER_AGENTS), các quy tắc viết lại URL và thiết đặt giá trị Content-Length
  - **CsrfViewMiddleware**: bổ sung khả năng bảo vệ chống lại CSRF
  - **AuthenticationMiddleware**: thêm thuộc tính user vào đối tượng request
  - **MessageMiddleware**: bổ sung hỗ trợ tin nhắn “flash”
  - **XFrameOptionsMiddleware**: bảo vệ chống lại các cuộc tấn công clickjacking vào trường tiêu đề X-Frame-Options

7.1 Xác thực người dùng

Middleware (Phần mềm trung gian)

- Các module được load theo trình tự khai báo trong danh sách của biến **MIDDLEWARE**
- Nên sử dụng tất cả các middleware mặc định của Django
- App Admin yêu cầu phải có **SessionMiddleware**, **AuthenticationMiddleware** và **MessageMiddleware**

7.1 Xác thực người dùng

Triển khai các view và template cho xác thực

- Django đã cung cấp các tính năng phục vụ xác thực trong app **authentication**, nhưng giao diện các trang của app này ở dạng mặc định

  → Sử dụng lại các view của app **authentication** và thực hiện một số thay đổi về giao diện giúp tiết kiệm nhiều thời gian và công sức

- Các nội dung cần thực hiện:
  - Sao chép các template phục vụ xác thực từ thư mục **django/contrib/admin /templates/admin/** sang thư mục **templates** của dự án
  - Thay đổi giao diện các template để thống nhất với giao diện dự án
  - Khai báo các ánh xạ URL để sử dụng các view của app **authentication** với các template đã điều chỉnh

7.1 Xác thực người dùng

Lưu trữ mật khẩu trên Django

- Mật khẩu được băm (hash) bằng các giải thuật băm (như PBKDF2/SHA256, BCrypt/SHA256 hay Argon2) trước khi được lưu vào CSDL
- Hàm băm: chuyển đổi dữ liệu 1 chiều

  → Tránh cho mật khẩu bị giải mã

  →Không ai có thể đọc được mật khẩu của người dùng
- Cấu trúc chuỗi giá trị lưu mật khẩu trong CSDL:
  <algorithm>$<iterations>$<salt>$<hash>
  - **<algorithm>:** Giải thuật băm
  - **<iterations>:** Số vòng lặp giải thuật
  - **<salt>:** Chuỗi ngẫu nhiên thêm vào mật khẩu trước khi băm
  - **<hash>:** Mật khẩu sau khi băm

7.1 Xác thực người dùng

Trang hồ sơ người dùng

Khi đăng nhập thành công, view **login** sẽ tự động chuyển đến trang /profile/ hiển thị thông tin hồ sơ người dùng sau khi đăng nhập thành công

- Nếu trang này không tồn tại sẽ xuất hiện trang báo lỗi **Page not Found**
- → Cần bổ sung thêm view và template cho trang này

Lưu ý: *Có thể điều chỉnh trang chuyển đến sau khi đăng nhập thành công bằng tham số next (**Ví dụ:** http://..../accounts/login/?**next=/accounts/profile2/**)*

Đối tượng request.user

- Mỗi đối tượng **request** luôn có thuộc tính **user**
- Nếu người dùng chưa xác thực (đăng nhập): user là **AnonymousUser**
- Nếu người dùng đã xác thực: user là đối tượng của lớp **User** chứa thông tin cá nhân của người dùng

7.1 Xác thực người dùng

Thay đổi giao diện theo người dùng

- Có thể sử dụng thông tin xác thực để áp dụng giao diện ứng dụng tùy theo tình trạng đăng nhập hay theo từng người dùng cụ thể
- Ngăn người dùng chưa đăng nhập truy cập view bằng decorator **login_required**
  - Tham số **login_url**: đường dẫn đến trang đăng nhập (mặc định lấy từ **settings.LOGIN_URL**)
  - Tham số **redirect_field_argument**: tên trường chứa URL để redirect (mặc định là next)

Ví dụ:

```
from django.contrib.auth.decorators import login_required
@login_required(login_url='/accounts/profile2')
def profile(request):
    …
@login_required(redirect_field_argument='redirect_to')
def more(request):
    …
```

7.1 Xác thực người dùng

Thay đổi giao diện theo người dùng

- Chỉ cho phép một người dùng hoặc nhóm người dùng cụ thể sử dụng một tính năng
  - Sử dụng decorator **permission_required**(<người dùng/ nhóm người dùng>)

    Ví dụ:

```
from django.contrib.auth.decorators import (login_required,
permission_required)
…

@permission_required('view_group')
def user_profile(request, uid):
    user = get_object_or_404(User, id=uid)
    permissions = user.get_all_permissions()
    return render(request, 'user_profile.html',
{'user': user, 'permissions': permissions})
```

7.1 Xác thực người dùng

Thay đổi giao diện theo người dùng

- Decorator tùy chỉnh
  - Cần định nghĩa một hàm tùy chỉnh
  - Sử dụng hàm tùy chỉnh làm tham số cho decorator **user_passes_test**

```
from django.contrib.auth.decorators import (login_required, permission_required,
user_passes_test)
…
def veteran_user(user):
    now = datetime.datetime.now()
    if user.date_joined is None:
        return False
    return now - user.date_joined > datetime.timedelta(days=365)

@user_passes_test(veteran_user)
def veteran_features(request):
    user = request.user
    permissions = user.get_all_permissions()
    return render(request, 'veteran_profile.html’,
        {'user': user, 'permissions': permissions})
```

7.2 Quản lý phiên

Giới thiệu

- Giao thức HTTP là giao thức không trạng thái (stateless)
  - Không có thông tin trạng thái được lưu lại máy chủ giữa các lần request
  - Khó khăn trong cung cấp các thông tin đặc thù cho từng người dùng
- Các ứng dụng web thích ứng với nhu cầu từng người dùng hiện nay triển khai lưu trữ trạng thái thông qua các phiên (session)
- Hoạt động của từng người dùng được lưu trữ thông qua một khóa phiên (session key)
  - Thông tin khóa phiên được lưu trữ lại và có thể truy xuất từ phía máy chủ
- Trên Django, các thao tác với phiên được triển khai dưới dạng một middleware.

7.2 Quản lý phiên

Bộ lưu trữ thông tin phiên

- Thông tin về các phiên được lưu trữ trên tập, CSDL, bộ nhớ hoặc cookie
- Middleware quản lý phiên của Django cho phép linh hoạt lựa chọn vị trí lưu thông tin phiên
  - Mặc định, Django lưu thông tin phiên trong CSDL tại máy chủ
  - Thiết đặt qua biến **SESSION_ENGINE** trong tập tin **settings.py**
- Các thiết đặt:
  - Lưu đệm (cached): lưu đệm thông tin phiên để phục vụ các ứng dụng hiệu năng cao có thể dùng **django.contrib.sessions.backends.cache** (lưu trong bộ nhớ) và **django.contrib.sessions.backends.cached_db** (lưu trong CSDL)
  - Lưu trên tập tin: dùng khi không ưu tiên về hiệu năng hoặc lý do khác
  - Lưu trên cookie: lưu thông tin trên máy khách

7.2 Quản lý phiên

Lưu trữ dưới dạng Pickle hay JSON

- **Pickle**
  - Thư viện cung cấp sẵn của Python
  - Cho phép tuần tự hóa các đối tượng trên Python thành chuỗi các byte nhị phân
  - Tương thích tốt với các phiên bản của Python và các hệ điều hành khác nhau
  - Tuy nhiên có các lỗ hổng bảo mật nên không phù hợp để lưu các thông không tin cậy
- **JSON (JavaScript Object Notation)**
  - Sử dụng cấu trúc của JS để lưu thông tin
  - Được dùng phổ biến trong trao đổi dữ liệu và thông điệp, đặc biệt tronng ứng dụng web
  - Cho phép tuần tự hóa dữ liệu thành chuỗi ký tự
  - Tuy nhiên không hỗ trợ dữ liệu kiểu ngày tháng → Cần chuyển thành dạng chuỗi ký tự trước khi sử dụng JSON

→ Django cung cấp sẵn 2 thư viện **PickleSerializer** và **JSONSerializer**

7.2 Quản lý phiên

Lưu trữ dữ liệu vào phiên

- Dữ liệu từ phiên có thể sử dụng để cải thiện trải nghiệm của người dùng
- Trên Django, dữ liệu phiên được lưu trong thuộc tính **session** của đối tượng **request**

  Ví dụ: `request.session['books_reviewed_count'] = 39`
- Một số lưu ý:
  - Khóa trong biến **session** phải là kiểu chuỗi ký tự
  - Các khóa bắt đầu bằng dấu gạch chân (underscore) được hiểu là giá trị sử dụng nội bộ hệ thống
  - Không được phép gán nguyên giá trị **request.session**, chỉ được phép truy xuất từng giá trị theo khóa

    Không thực hiện:
    ```
    request.session = {'books_read_count':30, 'books_reviewed_count': 39}
    ```
    Thay bằng:
    ```
    request.session['books_read_count'] = 30
    request.session['books_reviewed_count'] = 39
    ```

Hết chương 7